Uninformed (Blind) Search Algorithms

Prepared in the style of a Symbolic AI / Classical Search academic note. Includes: (i) one common connected example graph, (ii) pseudocode per algorithm, (iii) step-by-step run on the example, (iv) search tree construction, and (v) time/space complexity with a comparative analysis.

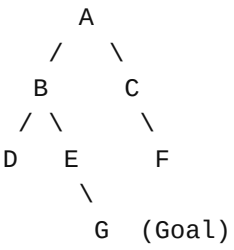
0. Common Example Graph (Used Throughout)

Start: A **Goal:** G

Adjacency (left-to-right order):

```
A: B, C
B: D, E
C: F
D: (none)
E: G
F: (none)
G: (none)
```

Graph sketch:



Edge costs: assumed unit cost (=1) unless explicitly stated otherwise. This makes BFS optimal and causes UCS to behave like BFS on this example.

Terminology clarification: “Best-first search” typically denotes a heuristic-driven (informed) strategy. Since this document covers *uninformed* search, we focus on BFS, DFS, DLS, IDS, and UCS. (If needed, best-first can be discussed as Greedy Best-First or A* under informed search.)

1. Breadth-First Search (BFS)

1.1 Core idea

Expands nodes in nondecreasing depth order (level by level) using a FIFO queue. With unit step costs, BFS is complete and optimal.

1.2 Pseudocode

```
BFS(start, goal):
    frontier ← FIFO queue
    frontier.enqueue(start)
```

```
    parent[start] ← null
    explored ← ∅

    while frontier not empty:
        n ← frontier.dequeue()
        if n == goal:
            return reconstruct_path(parent, n)
        explored.add(n)
        for each child in successors(n) in left-to-right order:
            if child not in explored and child not in frontier:
                parent[child] ← n
                frontier.enqueue(child)

    return failure
```

1.3 Step-by-step run on the example graph

Step	Frontier (Queue)	Expanded	Newly Enqueued
0	[A]	-	-
1	[B, C]	A	B, C
2	[C, D, E]	B	D, E
3	[D, E, F]	C	F
4	[E, F]	D	-
5	[F, G]	E	G
6	[G]	F	-
7	[]	G (goal)	-

Returned solution path: A → B → E → G

1.4 Search tree construction (BFS)

BFS constructs the search tree in layers. The moment G is discovered (as a child of E) it will be expanded after all earlier frontier nodes of equal or smaller depth are processed.

```
          [A]
         /   \
      [B]     [C]
     /   \       \
  [D]     [E]     [F]
           \
           [G]  ← goal found when expanded at depth 3
```

1.5 Complexity

Time: O(b^d) **Space:** O(b^d), where b is branching factor and d is depth of shallowest solution.

2. Depth-First Search (DFS)

2.1 Core idea

Expands the deepest unexplored node first (LIFO stack / recursion). DFS is memory-efficient but not optimal and can be incomplete in infinite-depth spaces.

2.2 Pseudocode

```
DFS(start, goal):
    frontier ← LIFO stack
    frontier.push(start)
    parent[start] ← null
    explored ← ∅

    while frontier not empty:
        n ← frontier.pop()
        if n == goal:
            return reconstruct_path(parent, n)
        if n in explored:
            continue
        explored.add(n)
        for each child in successors(n) in reverse left-to-right order:
            # reverse so that leftmost is processed first when popped
            if child not in explored:
                parent[child] ← n
                frontier.push(child)

    return failure
```

2.3 Step-by-step run on the example graph

Assume left-to-right preference: A expands B before C; B expands D before E.

Step	Frontier (Stack top at right)	Expanded	Newly Pushed
0	[A]	-	-
1	[C, B]	A	C, B
2	[C, E, D]	B	E, D
3	[C, E]	D	-
4	[C, G]	E	G
5	[C]	G (goal)	-

Returned solution path: A → B → E → G

Note: C remains unexpanded at termination (DFS can find a solution without exploring peers).

2.4 Search tree construction (DFS)

```
          [A]
         /
      [B]
     /   \
  [D]     [E]
           \
           [G]  ← goal found; subtree under C is not explored
```

2.5 Complexity

Time: O(b^m) **Space:** O(b·m), where m is maximum depth (or depth limit if imposed).

3. Depth-Limited Search (DLS)

3.1 Core idea

DFS with a fixed cutoff L. Prevents infinite descent but may miss solutions deeper than L.

3.2 Pseudocode

```
DLS(node, goal, limit):
    if node == goal:
        return success
    if limit == 0:
        return cutoff

    cutoff_occurred ← false
    for each child in successors(node) in left-to-right order:
        result ← DLS(child, goal, limit - 1)
        if result == cutoff:
            cutoff_occurred ← true
        else if result == success:
            return success

    if cutoff_occurred:
        return cutoff
    else:
        return failure
```

3.3 Step-by-step run (Limit L = 2)

Goal G is at depth 3, so with limit 2 the search terminates with a cutoff/failure (depending on reporting convention).

```
Depth 0: A
Depth 1: B, C
Depth 2: D, E, F
Stop (limit reached). G is not reachable within depth 2.
```

3.4 Search tree construction (DLS, L=2)

```
          [A]
         /   \
      [B]     [C]
     /   \      \
  [D]     [E]    [F]

(limit reached; G is beyond the cutoff)
```

3.5 Complexity

Time: O(b^L) **Space:** O(b·L)

4. Iterative Deepening Search (IDS)

4.1 Core idea

Runs DLS repeatedly with increasing limits (0,1,2,...). Achieves BFS-like optimality (unit costs) with DFS-like space usage. The repeated expansions are the price paid for low memory.

4.2 Pseudocode

```
IDS(start, goal):
    for depth = 0, 1, 2, ...:
        result ← DLS(start, goal, depth)
        if result == success:
            return solution
        # if failure (no cutoff and no success), can return failure for
finite graphs
```

4.3 Step-by-step run on the example graph

Iteration with limit 0

```
[A]
```

Iteration with limit 1

```
     [A]
    /   \
  [B]    [C]
```

Iteration with limit 2

```
         [A]
        /   \
     [B]     [C]
    /   \      \
  [D]    [E]    [F]
```

Iteration with limit 3 (goal discovered)

```
         [A]
        /   \
     [B]     [C]
    /   \      \
  [D]    [E]    [F]
          \
          [G]  ← goal found at depth 3
```

Returned solution path: A → B → E → G

4.4 Complexity

Time: O(b^d) **Space:** O(b·d)

5. Uniform-Cost Search (UCS)

5.1 Core idea

Expands the node on the frontier with the lowest path cost g(n) using a priority queue. UCS is complete and optimal for positive step costs. On unit-cost graphs it behaves like BFS.

5.2 Pseudocode

```
UCS(start, goal):
    frontier ← priority queue ordered by path_cost g
    frontier.insert(start, priority=0)
    parent[start] ← null
    best_g[start] ← 0

    while frontier not empty:
        n ← frontier.remove_min()
        if n == goal:
            return reconstruct_path(parent, n)

        for each child in successors(n):
            new_g ← best_g[n] + cost(n, child)
            if child not in best_g OR new_g < best_g[child]:
                best_g[child] ← new_g
                parent[child] ← n
                frontier.insert_or_decrease_key(child, priority=new_g)

    return failure
```

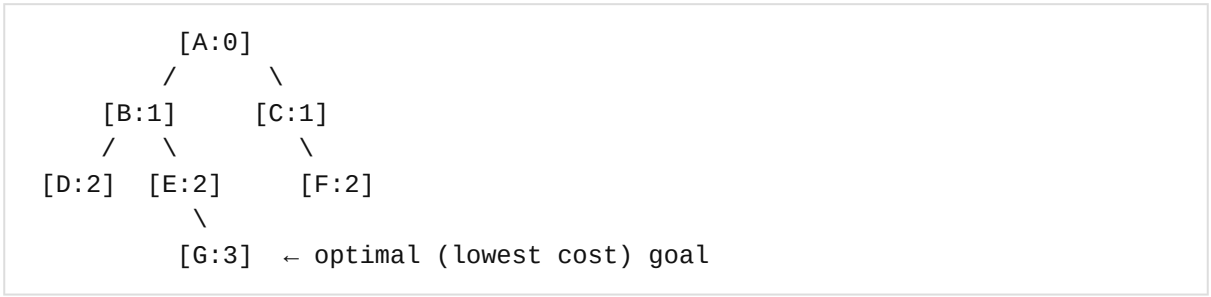
5.3 Step-by-step run (unit-cost edges)

Because all costs are 1, UCS expands by increasing depth, matching BFS's effective order on this example.

Step	Frontier (node: g)	Expanded	Notes
0	{A:0}	-	-
1	{B:1, C:1}	A	Insert B,C with cost 1
2	{C:1, D:2, E:2}	B	Insert D,E with cost 2
3	{D:2, E:2, F:2}	C	Insert F with cost 2
4	{E:2, F:2}	D	-
5	{F:2, G:3}	E	Insert G with cost 3
6	{G:3}	F	-
7	{}	G (goal)	Return path

Returned solution path: A → B → E → G

5.4 Search tree construction (UCS on unit costs)



5.5 Complexity

Time: O(b^(C*/ε)) **Space:** O(b^(C*/ε))

Where C* is the optimal solution cost and ε is the minimum positive step cost.

6. Comparative Analysis (Time & Space)

Algorithm	Complete?	Optimal?	Time Complexity	Space Complexity	Practical Notes
BFS	Yes (finite b)	Yes (unit costs)	O(b^d)	O(b^d)	Finds shallowest solution; memory-heavy.
DFS	No (can fail in infinite-depth)	No	O(b^m)	O(b·m)	Low memory; can get trapped down deep branches.
DLS	Only if L ≥ d	No (generally)	O(b^L)	O(b·L)	Controls DFS; trades completeness for bounded effort.
IDS	Yes (finite b)	Yes (unit costs)	O(b^d)	O(b·d)	Best general-purpose uninformed choice when memory is limited.
UCS	Yes (positive costs)	Yes	O(b^(C*/ε))	O(b^(C*/ε))	Needed when costs vary; reduces to BFS on unit costs.

6.1 High-level conclusions

- **Memory vs optimality:** BFS is optimal (unit cost) but can be infeasible in space; DFS uses little space but sacrifices guarantees.
- **IDS is the standard compromise:** BFS-level solution quality with DFS-level memory, at the cost of re-expanding upper levels.
- **Cost sensitivity:** When step costs differ, **UCS** is the correct uninformed algorithm for optimal solutions.

End of document.